



Signal and Image Processing MILSTONE 3

By:

(auto team)

mohamed Abdelhamid 13002323 T02

Haroun Ghoneim 13005136 T02

Andrew Elkess 13002881 T02

Rana Nagy 13007179 T08

Contents

Signal and Image Processing	1
GitHub link:	3
Milestone 3: CNN Implementation.....	3
1. CNN Pipeline and Architecture.....	3
2. Model Evaluation	4
3. Total Runtime	6
4. Top 3 Probabilities	6
Milestone 2: Previous Implementation.....	7
1. Updated Pipeline.....	7
2. Output Feature Vector	7
3. Confusion Matrix.....	8
4. Total Runtime	8

GitHub link:

<https://github.com/medo-abdelhamid/Image-milestone-2-project.git>

Milestone 3: CNN Implementation

1. CNN Pipeline and Architecture

For this milestone, I built a Convolutional Neural Network using PyTorch to classify the ball images into the six different categories. To help the model generalize and prevent overfitting, I applied some data augmentation to the training set, including random horizontal flips, 15-degree rotations, and color jitter. All images were resized to 128x128 pixels.

The network is split into two main parts: the convolutional feature extractor and the fully connected classifier.

Convolutional Layers: The feature extractor consists of 6 convolutional blocks. Each block follows the same basic pattern: a 2D Convolution, Batch Normalization, a ReLU activation, and a Max Pooling layer to downsample the spatial dimensions.

- **Block 1:** 3 input channels to 32 filters.
- **Block 2:** 32 filters to 64 filters.
- **Block 3:** 64 filters to 128 filters.
- **Block 4:** 128 filters to 256 filters.
- **Block 5:** 256 filters to 512 filters.
- **Block 6:** 512 filters to 512 filters. Instead of Max Pooling, this final block uses an Adaptive Average Pooling layer (1, 1) to ensure the output is perfectly sized for the flat linear layers regardless of slight input variations.

Fully Connected (Dense) Layers: After flattening the output from the conv blocks, the data passes through the classifier section:

- A Linear layer reducing the 512 features down to 128.
- A ReLU activation.
- A Dropout layer set to 0.8. This high dropout rate forces the network to not rely on any single node too heavily.
- A final Linear output layer mapping to our 6 classes (tennis ball, golf ball, football, basketball, baseball, American football).

Training Parameters: The model was trained for 30 epochs using the Adam optimizer (learning rate = 0.001, weight decay = 1e-4). I also used a Cosine Annealing Learning Rate Scheduler to smoothly adjust the learning rate during the training process, alongside a standard Cross-Entropy Loss function.

2. Model Evaluation

The model performed exceptionally well on the test dataset, missing almost zero classifications. The overall metrics are:

- **Accuracy:** 99.31%
- **Precision (Weighted):** 0.9932
- **Recall (Weighted):** 0.9931

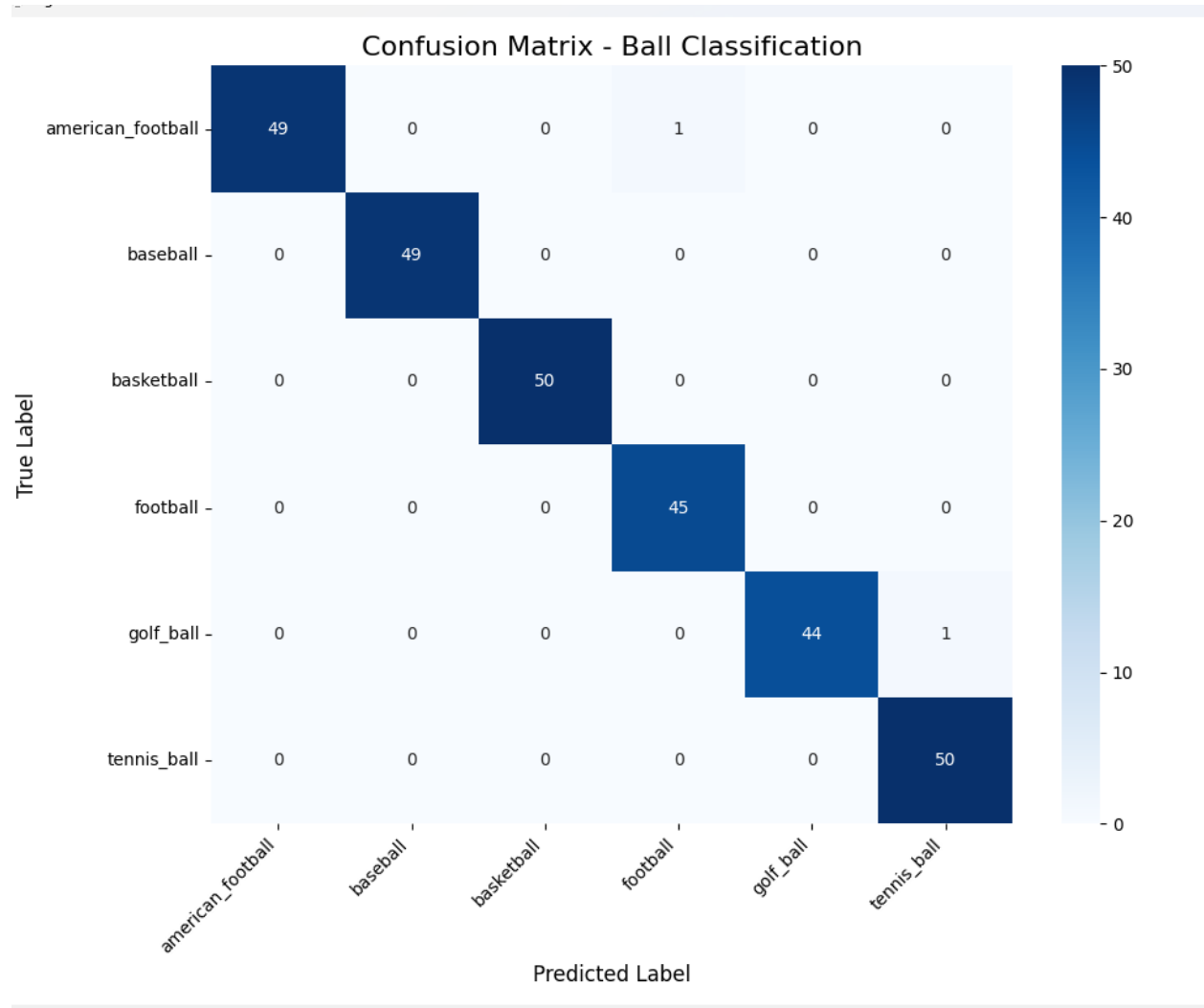
```
Starting Training for 30 epochs...
Epoch 1/30 - Loss: 1.1963
Epoch 2/30 - Loss: 0.7669
Epoch 3/30 - Loss: 0.6819
Epoch 4/30 - Loss: 0.6409
Epoch 5/30 - Loss: 0.5613
Epoch 6/30 - Loss: 0.5676
Epoch 7/30 - Loss: 0.4487
Epoch 8/30 - Loss: 0.4304
Epoch 9/30 - Loss: 0.4719
Epoch 10/30 - Loss: 0.4581
Epoch 11/30 - Loss: 0.4027
Epoch 12/30 - Loss: 0.3302
Epoch 13/30 - Loss: 0.2654
Epoch 14/30 - Loss: 0.2784
Epoch 15/30 - Loss: 0.3035
Epoch 16/30 - Loss: 0.2820
Epoch 17/30 - Loss: 0.2397
Epoch 18/30 - Loss: 0.2517
Epoch 19/30 - Loss: 0.2160
Epoch 20/30 - Loss: 0.1893
Epoch 21/30 - Loss: 0.1986
Epoch 22/30 - Loss: 0.1554
Epoch 23/30 - Loss: 0.1704
Epoch 24/30 - Loss: 0.1378
Epoch 25/30 - Loss: 0.1437
Epoch 26/30 - Loss: 0.1392
Epoch 27/30 - Loss: 0.1344
Epoch 28/30 - Loss: 0.1296
Epoch 29/30 - Loss: 0.1110
Epoch 30/30 - Loss: 0.1232

Total Runtime for Training: 14.59 minutes

Starting Evaluation...
Accuracy: 99.31%
Precision (Weighted): 0.9932
Recall (Weighted): 0.9931
Confusion Matrix:
[[49  0  0  1  0  0]
 [ 0 49  0  0  0  0]
 [ 0  0 50  0  0  0]
 [ 0  0  0 45  0  0]
 [ 0  0  0  0 44  1]
 [ 0  0  0  0  0 50]]
```

Auto team

Confusion Matrix



3. Total Runtime

The model was trained over 30 epochs, and the total runtime for training was **14.59 minutes**.

4. Top 3 Probabilities

When scanning the entire test set to find the predictions with the highest confidence, the model was 100% certain about these top three examples:

1. **Predicted:** baseball (100.0000%) | **True Label:** baseball
2. **Predicted:** football (100.0000%) | **True Label:** football
3. **Predicted:** baseball (100.0000%) | **True Label:** baseball

```
Scanning the entire test set for the top 3 best predictions
```

```
TOP 3
```

- ```
1. Predicted: baseball (100.0000%) | True Label: baseball
2. Predicted: football (100.0000%) | True Label: football
3. Predicted: baseball (100.0000%) | True Label: baseball
```

## Milestone 2: Previous Implementation

### 1. Updated Pipeline

In Milestone 2, the pipeline relied on traditional computer vision techniques rather than deep learning. During optimization, I found that an image resize of 128x128 lost too much detail, so the resolution was increased to 256x256, which significantly improved the clarity of texture features.

For the classification model, I initially experimented with SVM and k-NN, but their accuracy was quite low. Because the combined feature vector was very large and complex, I ultimately updated the pipeline to use a Random Forest classifier with 500 trees, which handled the complexity much better and yielded a final accuracy of 95.85%.

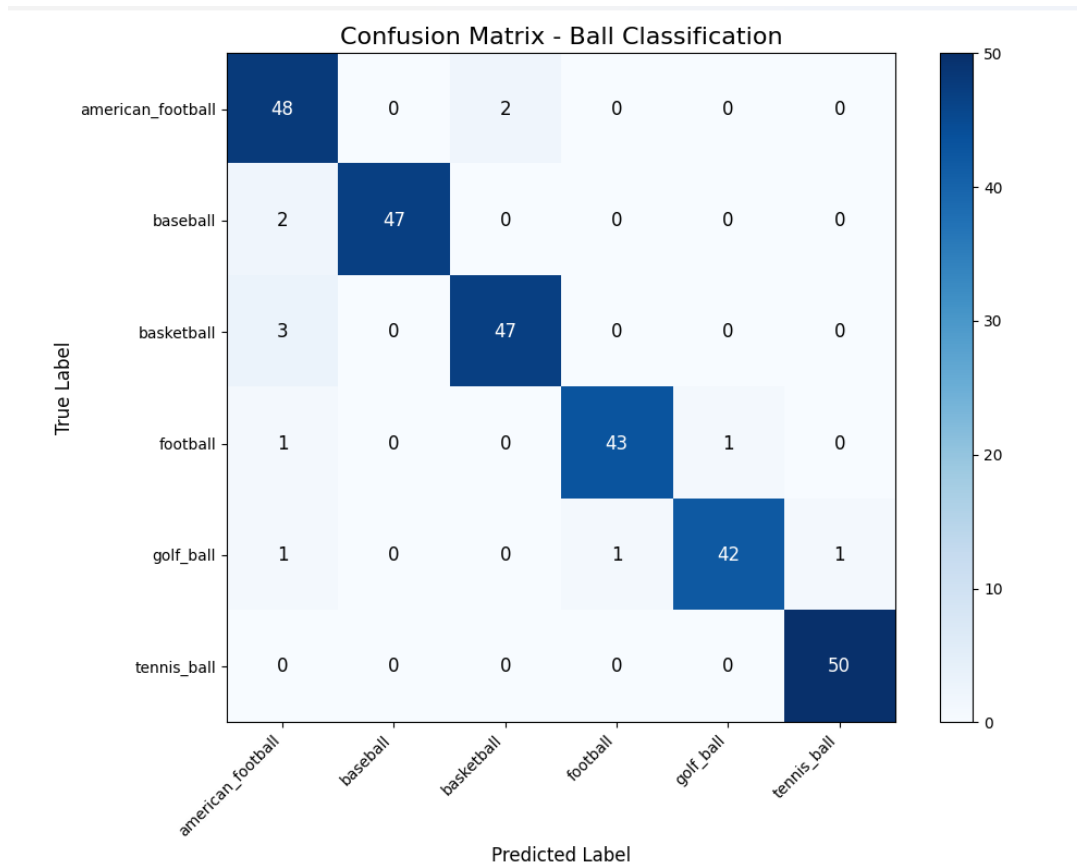
### 2. Output Feature Vector

Instead of the CNN automatically learning features, I manually engineered a comprehensive feature vector to capture every detail of the balls. The final output feature vector is a combination of the following extractions:

- **Color Histograms:** 3D RGB and 1D grayscale histograms to capture primary colors.
- **GLCM (Gray-Level Co-occurrence Matrix):** To identify surface textures through contrast, dissimilarity, homogeneity, energy, and correlation.
- **HOG (Histograms of Oriented Gradients):** To capture overall structure and directional edges.
- **SIFT (Scale Invariant Feature Transform):** Tuned specifically to 65 keypoints to find unique features regardless of rotation.
- **Contour Features:** Calculating the number of contours, area, and circularity.
- **DoG (Difference of Gaussians):** Used to highlight fine edges and missed details.
- **K-Means Features:** Clustering pixel values to find dominant colors and their proportions.
- **Hough Circle Transform:** Specifically added to detect the radius and center, giving the model a hint about the circular shape and size.

### 3. Confusion Matrix

Here is the confusion matrix for the Random Forest model from Milestone 2, which achieved an overall accuracy of 95.85%



### 4. Total Runtime

The whole process took somewhere between 3 to 5 minutes to complete.